PROYECTO FINAL BACKEND · SOLUCIONES PARA EL CLENA

Modelo de datos del sistema Conserva

P3 · Monitoreo y alertas de conservación del Archivo Histórico
Diagrama entidad-relación, diccionario de datos, índices y datos personales — Issue #23

Proyecto	P3 · Monitoreo y alertas de conservación (equipo de origen: G7)
Autor	Víctor Cantillo Valencia (Development Team)
Equipo	Gabriel Caicedo de la Iglesia (Product Owner) Leonel José Triana Martínez (Scrum Master) Víctor Cantillo Valencia (Development Team) Sebastián González Quintanilla (Development Team)
Motor	PostgreSQL 17 · TypeORM 0.3 (EntitySchema) · migraciones versionadas
Curso	Backend · Ingeniería de Sistemas · Universidad del Norte
Repositorio	github.com/mogeko-cpu/Proyecto-Final-Backend
Fecha	12 de noviembre de 2026
Versión	1.0

Índice

1. Propósito y alcance 3

2. Diagrama entidad-relación 4

3. Diccionario de datos 5

4. Relaciones y cardinalidades 9

5. Reglas de integridad 10

6. Índices y su justificación 11

7. Datos personales mínimos 13

8. Migraciones y evolución del esquema 14

1. Propósito y alcance

Este documento describe la base de datos de Conserva, el sistema de monitoreo y alertas de conservación del Archivo Histórico del CLENA. Corresponde al issue #23 del Sprint 1 y refleja exactamente el esquema que crean las migraciones del repositorio (`backend/src/infrastructure/migrations/`); si el código y este documento no coinciden, manda el código.

El modelo tiene **ocho tablas** agrupadas en cuatro módulos:

Módulo	Tablas	Historias de usuario
Autenticación y usuarios	`users`, `sessions`	Registro, inicio de sesión, roles y eliminación de cuenta
Inventario	`deposits`, `records`	HU03 a HU05: depósitos con rangos, expedientes, archivado
Monitoreo	`inspections`, `readings`, `alerts`	HU06 a HU10: inspecciones, lecturas, alertas y vencimientos
Intervenciones	`tasks`	HU12 a HU14: tareas de atención y dashboard

Convenciones

- Todas las claves primarias son `uuid` generados por la aplicación; así los identificadores no revelan cuántos registros existen ni el orden de creación.
- Toda tabla tiene `createdAt` de tipo `timestamptz`; las fechas se guardan con zona horaria y la interfaz las muestra en la hora local del navegador con formato es-CO.
- Los nombres de columnas siguen el estilo camelCase de las entidades del dominio (en SQL se escriben entre comillas).
- Los valores enumerados se guardan como `varchar` con `CHECK`, en lugar de tipos `ENUM`, para poder agregar valores con una migración simple.
- No hay borrados físicos de datos de negocio: los expedientes se archivan y las cuentas se anonimizan.

2. Diagrama entidad-relación

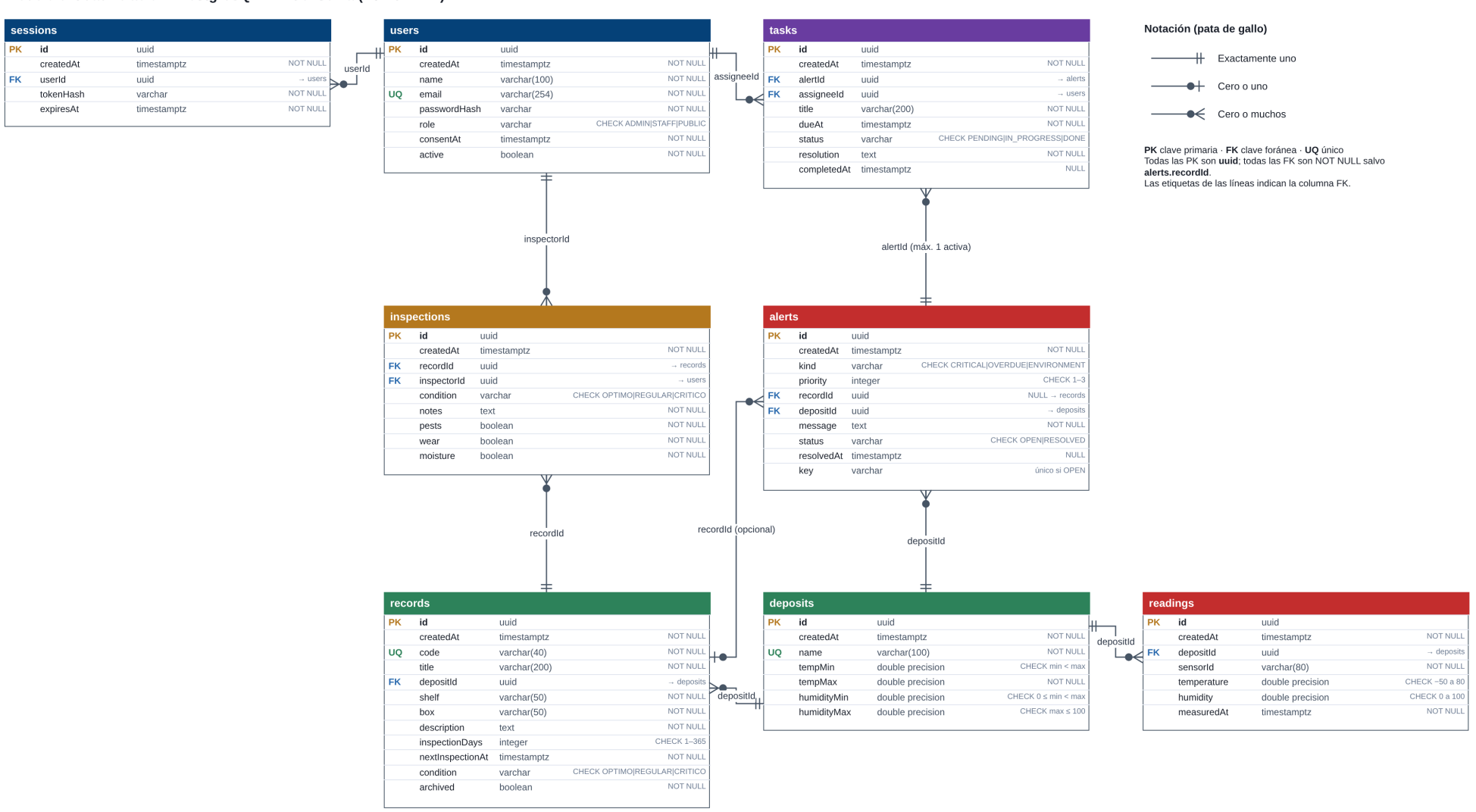


Figura 1. Modelo entidad-relación en notación pata de gallo. Fuente editable: `docs/diagramas/modelo-datos.drawio`.

3. Diccionario de datos

Columnas de cada tabla tal como las crea la migración `Initial`. Las columnas con el mismo tipo y propósito se agrupan con una barra (por ejemplo `tempMin / tempMax`).

`users`

Personas que usan el sistema. Se crean por registro público (rol PUBLIC) o por un ADMIN.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador generado por la aplicación.
`createdAt`	timestamptz	No		Fecha de alta.
`name`	varchar(100)	No		Nombre para mostrar.
`email`	varchar(254)	No	UQ	Correo de inicio de sesión, guardado en minúsculas.
`passwordHash`	varchar	No		Hash bcrypt (costo 12). Nunca se devuelve en la API.
`role`	varchar	No		ADMIN, STAFF o PUBLIC.
`consentAt`	timestamptz	No		Momento en que aceptó el tratamiento de datos.
`active`	boolean	No		Falso si la cuenta fue eliminada (anonimizada).

`sessions`

Sesiones de refresh activas; permiten rotar y revocar tokens.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador de la sesión (va dentro del refresh token).
`createdAt`	timestamptz	No		Fecha de creación.
`userId`	uuid	No	FK → users	Dueño de la sesión.
`tokenHash`	varchar	No		Digest SHA-256 del refresh token; el token no se guarda.
`expiresAt`	timestamptz	No		Vencimiento (7 días).

`deposits`

Depósitos del archivo con sus rangos ambientales aceptables.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador.
`createdAt`	timestamptz	No		Fecha de alta.
`name`	varchar(100)	No	UQ	Nombre del depósito.
`tempMin / tempMax`	double precision	No		Rango de temperatura en °C (mín. < máx.).
`humidityMin / humidityMax`	double precision	No		Rango de humedad relativa en % (0 ≤ mín. < máx. ≤ 100).

`records`

Expedientes o unidades documentales ubicadas en un depósito.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador.
`createdAt`	timestamptz	No		Fecha de alta.
`code`	varchar(40)	No	UQ	Signatura del expediente (por ejemplo EXP-001).
`title`	varchar(200)	No		Título descriptivo.
`depositId`	uuid	No	FK → deposits	Depósito donde se ubica.
`shelf / box`	varchar(50)	No		Estante y caja.
`description`	text	No		Descripción libre.
`inspectionDays`	integer	No		Periodicidad de inspección, de 1 a 365 días.
`nextInspectionAt`	timestamptz	No		Próxima inspección; al crear queda pendiente de inmediato.
`condition`	varchar	No		Estado según la última inspección: OPTIMO, REGULAR o CRITICO.
`archived`	boolean	No		Baja lógica: conserva el historial y bloquea la edición.

inspections

Historial de inspecciones físicas de un expediente.

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador.
createdAt	timestamptz	No		Fecha de la inspección.
recordId	uuid	No	FK → records	Expediente inspeccionado.
inspectorId	uuid	No	FK → users	Usuario que inspeccionó.
condition	varchar	No		OPTIMO, REGULAR o CRITICO.
notes	text	No		Observaciones.
pests / wear / moisture	boolean	No		Presencia de plagas, deterioro y humedad.

readings

Lecturas ambientales enviadas por los sensores (o el simulador).

Columna	Tipo	Nulo	Clave	Descripción
id	uuid	No	PK	Identificador.
createdAt	timestamptz	No		Fecha de registro en el sistema.
depositId	uuid	No	FK → deposits	Depósito medido.
sensorId	varchar(80)	No		Identificador del sensor.
temperature	double precision	No		°C, entre −50 y 80.
humidity	double precision	No		%, entre 0 y 100.
measuredAt	timestamptz	No		Momento de la medición (últimas 24 h, no futura).

`alerts`

Alertas de conservación generadas por reglas del dominio.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador.
`createdAt`	timestamptz	No		Fecha de apertura.
`kind`	varchar	No		CRITICAL (inspección crítica), OVERDUE (inspección vencida) o ENVIRONMENT (lectura fuera de rango).
`priority`	integer	No		1 (más urgente) a 3.
`recordId`	uuid	Sí	FK → records	Expediente afectado; nulo en alertas ambientales.
`depositId`	uuid	No	FK → deposits	Depósito afectado.
`message`	text	No		Texto legible para el personal.
`status`	varchar	No		OPEN o RESOLVED.
`resolvedAt`	timestamptz	Sí		Fecha de cierre.
`key`	varchar	No		Clave de deduplicación, por ejemplo `OVERDUE:<recordId>`.

`tasks`

Tareas de intervención asignadas para atender una alerta.

Columna	Tipo	Nulo	Clave	Descripción
`id`	uuid	No	PK	Identificador.
`createdAt`	timestamptz	No		Fecha de creación.
`alertId`	uuid	No	FK → alerts	Alerta que origina la tarea.
`assigneeId`	uuid	No	FK → users	Responsable.
`title`	varchar(200)	No		Título.
`dueAt`	timestamptz	No		Fecha límite.
`status`	varchar	No		PENDING → IN_PROGRESS → DONE (un paso a la vez).
`resolution`	text	No		Descripción de lo realizado (obligatoria al cerrar).
`completedAt`	timestamptz	Sí		Fecha de cierre.

4. Relaciones y cardinalidades

Todas las relaciones son de uno a muchos y se implementan con claves foráneas que apuntan a la clave primaria de la tabla padre. No hay relaciones muchos a muchos.

Relación	Cardinalidad	Clave foránea	Significado
users → sessions	1 : 0..N	`sessions.userId`	Un usuario puede tener varias sesiones abiertas (varios dispositivos); cada sesión pertenece a un usuario.
users → inspections	1 : 0..N	`inspections.inspectorId`	Cada inspección registra quién la hizo.
users → tasks	1 : 0..N	`tasks.assigneeId`	Un usuario puede ser responsable de varias tareas.
deposits → records	1 : 0..N	`records.depositId`	Cada expediente está en exactamente un depósito.
deposits → readings	1 : 0..N	`readings.depositId`	Serie temporal de lecturas por depósito.
deposits → alerts	1 : 0..N	`alerts.depositId`	Toda alerta queda asociada a un depósito, también las de expediente.
records → inspections	1 : 0..N	`inspections.recordId`	Historial de inspecciones del expediente.
records → alerts	0..1 : 0..N	`alerts.recordId` (nulable)	Las alertas CRITICAL y OVERDUE apuntan a un expediente; las ENVIRONMENT no.
alerts → tasks	1 : 0..N	`tasks.alertId`	Una alerta puede tener varias tareas en su historia, pero como máximo una no terminada (índice único parcial).

Comportamiento al borrar

Ninguna clave foránea usa `ON DELETE CASCADE`: PostgreSQL rechaza borrar un padre que tenga hijos. Es intencional, porque el historial de conservación (inspecciones, alertas, tareas) es evidencia institucional y no debe desaparecer por un borrado accidental. Por eso:

- `DELETE /api/v1/records/:id` no borra: marca `archived = true`, resuelve las alertas abiertas del expediente y bloquea la edición (HU05).
- `DELETE /api/v1/auth/account` no borra al usuario: lo anonimiza y elimina solo sus sesiones (sección 7).

5. Reglas de integridad

Las reglas se validan en dos lugares: en el dominio (`domain/rules/`), que devuelve mensajes claros al usuario, y en la base de datos, que es la última barrera si algún camino del código se salta una validación. Si la base rechaza una operación por una restricción, la API responde 400 con el mensaje «Los datos incumplen una regla de integridad», sin exponer detalles internos.

Tabla	Restricciones
`users`	`email` UNIQUE; `role` ∈ {ADMIN, STAFF, PUBLIC}
`deposits`	`name` UNIQUE; `tempMin < tempMax`; `humidityMin < humidityMax`; `humidityMin ≥ 0` y `humidityMax ≤ 100`
`records`	`code` UNIQUE; `inspectionDays` entre 1 y 365; `condition` ∈ {OPTIMO, REGULAR, CRITICO}
`inspections`	`condition` ∈ {OPTIMO, REGULAR, CRITICO}
`readings`	`temperature` entre −50 y 80; `humidity` entre 0 y 100
`alerts`	`kind` ∈ {CRITICAL, OVERDUE, ENVIRONMENT}; `priority` entre 1 y 3; `status` ∈ {OPEN, RESOLVED}; índice único parcial sobre `key` cuando `status = 'OPEN'`
`tasks`	`status` ∈ {PENDING, IN_PROGRESS, DONE}; índice único parcial sobre `alertId` cuando `status <> 'DONE'`

Reglas que dependen de varias filas

Dos reglas no se pueden expresar con un `CHECK` porque comparan filas distintas. Se resuelven con **índices únicos parciales**:

- **Una sola alerta abierta por situación.** Cada alerta lleva una clave como `OVERDUE:<recordId>` o `ENVIRONMENT:<depositId>`. El índice `alerts_open_key_idx` impide dos filas abiertas con la misma clave, aunque el temporizador de vencidas y una petición manual corran al mismo tiempo. Al resolverse, la clave queda libre para una alerta futura.
- **Una sola tarea activa por alerta.** El índice `tasks_active_alert_idx` solo cubre tareas no terminadas, así que una alerta puede acumular tareas cerradas en su historia pero nunca dos en curso.

Además, las operaciones que tocan varias tablas (registrar una inspección y abrir una alerta, archivar un expediente y resolver sus alertas) se ejecutan dentro de una transacción.

6. Índices y su justificación

PostgreSQL crea automáticamente un índice para cada clave primaria y cada restricción `UNIQUE` (`users.email`, `deposits.name`, `records.code`). Los índices adicionales están en las dos migraciones y cada uno responde a una consulta concreta de la API:

Índice	Definición	Consulta que acelera y por qué
`sessions_user_idx`	`sessions(userId)`	Revocar todas las sesiones de un usuario al eliminar la cuenta (`DELETE /auth/account`). Sin él, cada borrado recorre toda la tabla.
`records_deposit_idx`	`records(depositId)`	Filtro `GET /records?depositId=` y verificación de la FK al validar depósitos.
`records_due_idx`	`records(nextInspectionAt) WHERE NOT archived`	Parcial: solo indexa expedientes vigentes, que son los únicos que pueden vencer. Pensado para la evaluación de vencidas (HU09) y para ordenar por `nextInspectionAt`. Hoy `EvaluateOverdue` filtra el vencimiento en la aplicación; el índice queda listo para mover ese filtro a SQL cuando crezca el volumen.
`records_condition_idx`	`records(condition)`	Filtro `GET /records?status=CRITICO` (el parámetro `status` se traduce a `condition`).
`records_title_idx`	`records(lower(title) text_pattern_ops)`	Búsquedas por prefijo sin distinguir mayúsculas. Es complementario: la búsqueda actual (`ILIKE '%q%'`) la resuelve el índice trigram.
`inspections_history_idx`	`inspections(recordId, createdAt DESC)`	Historial de un expediente ordenado de la más reciente a la más antigua (`GET /records/:id/inspections`) y consulta de la última inspección al editar.
`readings_deposit_date_idx`	`readings(depositId, measuredAt DESC)`	Última lectura de un depósito al registrar una nueva (`POST /readings`) y listados por depósito. Es la tabla que más crece.
`alerts_open_key_idx`	`UNIQUE alerts(key) WHERE status='OPEN'`	Garantiza en la base que no haya dos alertas abiertas iguales aunque dos procesos evalúen a la vez. También acelera la búsqueda de la alerta abierta antes de crearla.
`alerts_priority_idx`	`alerts(status, priority, createdAt)`	Bandeja de alertas abiertas ordenadas por prioridad y antigüedad (`GET /alerts`, dashboard).
`tasks_active_alert_idx`	`UNIQUE tasks(alertId) WHERE status<>'DONE'`	Regla de negocio: como máximo una tarea activa por alerta.

`tasks_assignee_idx`	`tasks(assigneeId)`	"Mis tareas": tareas de un responsable.
`tasks_status_due_idx`	`tasks(status, dueAt)`	Tareas pendientes ordenadas por fecha límite.
`users_role_idx`	`users(role) WHERE active`	Contar administradores activos para impedir eliminar al último (`DeleteAccount`) y filtrar `GET /users?role=`.
`*_search_idx (GIN trigram, 10 índices)`	`records(code, title)`, `deposits(name)`, `users(name, email)`, `inspections(notes)`, `readings(sensorId)`, `alerts(message)`, `tasks(title)`	El parámetro `q` de los listados busca con `ILIKE '%q%'` en estas columnas. Un B-tree no sirve para comodines al inicio; la extensión `pg_trgm` sí.

Criterios usados

- **Índices compuestos en el orden de la consulta:** primero la columna de igualdad (`recordId`, `depositId`, `status`) y después la de orden (`createdAt`, `measuredAt`, `dueAt`), para que PostgreSQL devuelva las filas ya ordenadas sin un paso de ordenamiento.
- **Índices parciales** donde solo interesa una parte de la tabla (expedientes no archivados, alertas abiertas, tareas activas, usuarios activos): son más pequeños y además sirven para reglas de unicidad.
- **Ordenamiento validado:** la API solo permite ordenar por columnas de una lista blanca por tabla (por ejemplo, en `records`: `createdAt`, `code`, `title`, `nextInspectionAt`, `condition`), lo que evita inyección SQL y ordenamientos sobre columnas sin sentido.
- **Sin índices de más:** cada índice adicional encarece las escrituras. En `readings`, la tabla con más inserciones, solo hay dos índices además de la clave primaria.

7. Datos personales mínimos

El sistema maneja datos personales de los usuarios y debe cumplir la Ley 1581 de 2012 (protección de datos personales en Colombia). El criterio es guardar lo mínimo necesario para operar.

Qué se guarda y para qué

Dato	Finalidad	Protección
`users.name`	Identificar quién inspeccionó o es responsable de una tarea.	Solo visible para usuarios autenticados con rol ADMIN o STAFF.
`users.email`	Inicio de sesión.	Se normaliza a minúsculas; nunca aparece en listados públicos.
`users.passwordHash`	Verificar la contraseña.	Hash bcrypt con costo 12 y sal propia; la contraseña nunca se guarda ni se registra en logs. La API nunca devuelve este campo.
`users.consentAt`	Evidencia de la autorización del titular, exigida por la ley.	Se registra al crear la cuenta; el registro exige aceptar el tratamiento.
`sessions.tokenHash`	Validar y revocar refresh tokens.	Se guarda el digest SHA-256 del token, no el token: una copia de la base no permite iniciar sesión.

Qué no se guarda

No se piden documento de identidad, teléfono, dirección, fecha de nacimiento ni fotografías. Los logs registran método, ruta, estado, duración y un `requestId`, pero no cuerpos de petición ni tokens. Los datos de prueba y de demostración son ficticios; no se usan datos reales del CLENA.

Derecho de supresión

`DELETE /api/v1/auth/account` aplica el derecho del titular a que se supriman sus datos sin romper la trazabilidad de las inspecciones y tareas que hizo:

```
name         = 'Cuenta eliminada'
email        = '<id>@deleted.invalid'
passwordHash = ''
active       = false
sessions     → se borran todas las del usuario
```

Las inspecciones y tareas siguen apuntando al mismo `id`, pero ya no se puede saber quién era la persona. El dominio `.invalid` está reservado (RFC 2606), así que nunca corresponde a un correo real, y conservar `id` en el correo mantiene la restricción `UNIQUE`. Si la cuenta es la del último administrador activo, la operación se rechaza para no dejar el sistema sin administración.

8. Migraciones y evolución del esquema

El esquema solo cambia mediante migraciones de TypeORM revisadas en un pull request; la opción `synchronize` está desactivada. La API aplica las migraciones pendientes al iniciar, de modo que `docker compose up` deja la base lista en una máquina limpia.

Migración	Contenido
`1791417600000-Initial.ts`	Las ocho tablas con sus claves, restricciones `CHECK` y `UNIQUE`, y los índices de consulta e índices únicos parciales.
`1791417600001-SearchIndexes.ts`	Extensión `pg_trgm` y los diez índices GIN trigram para el parámetro de búsqueda `q`.

Cada migración tiene su método `down` para revertirla. Un cambio futuro se hace con una migración nueva; las ya aplicadas no se editan.

> **Mapeo con el código.** Las entidades del dominio (`domain/model.ts`) son interfaces TypeScript sin dependencias del ORM. El mapeo a estas tablas vive en `infrastructure/schema.ts` con `EntitySchema`, y el acceso se hace a través del puerto `Repository` (`domain/ports.ts`), implementado por `PgRepository` en `infrastructure/database.ts`.